

PROVISIONAL PATENT APPLICATION COVER SHEET

Based on USPTO Form SB/16 · 35 U.S.C. § 111(b) · 37 C.F.R. § 1.51(c)

☒ Provisional Patent Application (35 U.S.C. § 111(b))

System and Method for Personalized Name Generation Using Structured Preference Elicitation, Canonical Intent Mapping, Reaction-Driven Refinement, Multi-Provider Quality Routing, Taste State Maintenance, and Exploration-Driven Refinement

INVENTOR(S)

Normand

Daniel

A.

INVENTOR RESIDENCE

Temecula

CA

US

Daniel A. Normand
43997 Eaglebluff Court
Temecula, CA 92592
United States

☒ Micro Entity (37 C.F.R. § 1.29) ☐ Small Entity ☐ Large Entity

September 15, 2026

DAN-NE-001-PROV

DECLARATION / STATEMENT

The undersigned inventor(s) hereby declare that the attached specification discloses the invention described herein and that this application is being filed pursuant to 35 U.S.C. § 111(b) as a Provisional Patent Application. The inventor(s) acknowledge that a non-provisional application claiming the benefit of this provisional must be filed within twelve (12) months of this filing date.

SIGNATURE OF INVENTOR

Daniel A. Normand

DATE

September 15, 2026

ENCLOSURES

- ☒ Specification (including claims, abstract, and drawings)
- ☒ Figures: 9 formal drawings (FIG. 1 – FIG. 9)
- ☒ Filing fee (Micro Entity rate — to be submitted via USPTO Patent Center)

NamEngine — Provisional Patent Application

Prepared: 2026-09-15

Revised: 2026-09-15 — Claims restructured and expanded following defense review. Revised 2026-09-15 (v2) — Advanced Taste State and Exploration Planner embodiment added (Claims 15–17).

Inventor: Dan Normand

Status: DRAFT — Review before filing

■■ This document is a self-drafted provisional patent application intended to establish a USPTO priority date. It is not a substitute for advice from a registered patent attorney. Consider having a patent attorney review before filing the non-provisional.

PROVISIONAL PATENT APPLICATION

TITLE OF THE INVENTION

System and Method for Personalized Name Generation Using Structured Preference Elicitation, Canonical Intent Mapping, Reaction-Driven Refinement, Multi-Provider Quality Routing, Taste State Maintenance, and Exploration-Driven Refinement

FIELD OF THE INVENTION

The present invention relates to computer-implemented systems and methods for generating personalized names. More particularly, the invention relates to a multi-vertical naming platform that elicits qualitative user preferences through a structured sequential intake process, normalizes those preferences into a canonical naming intent representation, generates candidate names using a multi-provider artificial intelligence routing and quality scoring framework, and refines results across multiple rounds using reaction-derived taste profiles.

BACKGROUND OF THE INVENTION

Naming — whether for a newborn child, a pet, or a business — is one of the most personally significant decisions a person makes. Existing tools for name discovery suffer from fundamental limitations:

List-based search tools offer large databases filtered by simple criteria (origin, first letter, meaning keyword). They require users to already know what they are looking for, present results without personalization, and cannot reason about the emotional, stylistic, or cultural fit of a name for a specific person's situation.

Generic AI assistants (large language model chat interfaces) can suggest names in response to natural-language prompts, but they lack structured intake, produce inconsistent quality, do not persist user preferences across sessions, and provide no mechanism for iterative refinement based on user reactions.

Crowdsourced naming tools present popularity rankings and user-submitted favorites, but offer no mechanism to personalize results to an individual's unique aesthetic preferences, family context, or naming philosophy.

Certain commercial naming tools illustrate the limitations of the prior art. **Namify** and similar business-name generators may accept keyword or industry inputs and return generated brand or domain-oriented suggestions, but do not appear to normalize user preferences into a Canonical Naming Intent layer, do not appear to apply vertical-specific quality adapters across multiple naming domains, and do not appear to construct a session-chain taste profile from user reactions for use in later rounds. **Namelix** and similar AI-assisted brand-name tools may generate short brandable names from keywords and style constraints, but do not appear to provide a domain-independent canonical intent object, do not appear to score results through vertical-specific quality adapters for baby, pet, business, and other verticals, and do not appear to maintain a linked multi-round session chain from which demonstrated taste is reconstructed. **Squadhelp** and similar marketplace or contest-based naming platforms may combine human-submitted names, marketplace listings, or branding services, but do not appear to implement a CNI normalization layer, do not appear to use vertical-specific quality adapter scoring as a shared generation architecture, and do not appear to build and reuse a machine-readable taste profile across a parent-linked sequence of generation sessions. *Note to counsel: the foregoing characterizations of named third-party products are based on publicly available information as of the filing date and should be reviewed and softened as appropriate before filing the non-provisional application.*

Recommender systems, including media recommendation systems such as Netflix and Spotify, are also known in the art, as are wizard-style user interfaces that collect inputs through a sequence of screens. The invention does not claim recommender systems or wizard-style intake in isolation. Rather, the novelty resides in the specific computer-implemented combination of: structured naming preference elicitation; normalization into a Canonical Naming Intent object; vertical-specific quality adapter scoring; reaction-derived taste profile construction across a session chain; and reuse of that taste profile to drive subsequent AI naming rounds while excluding previously presented names.

None of these approaches:

- Systematically elicit qualitative preference signals through a guided, structured intake process before generation;
- Normalize those preferences into a stable intermediate representation suitable for driving AI generation across multiple providers;
- Build a machine-readable taste profile from user reactions to prior results;
- Route generation requests to multiple AI providers and select the highest-quality candidates using vertical-specific scoring;
- Apply different quality frameworks to different naming domains (baby names, pet names, business names) while sharing a common architectural foundation.

There is therefore a need for a system that treats naming as a taste-driven, iterative discovery process rather than a keyword search, and that applies structured AI orchestration to produce personalized, high-quality name candidates tailored to each user's specific situation and aesthetic preferences.

SUMMARY OF THE INVENTION

The present invention provides a computer-implemented system and method — referred to herein as the **NamEngine platform** — for generating personalized name candidates across multiple naming domains through a structured preference elicitation, canonical intent mapping, AI generation routing, quality scoring, and reaction-driven refinement pipeline.

In one aspect, the invention provides a **structured sequential preference elicitation system** that presents intake questions to a user one at a time, builds a named "direction profile" from responses, presents that direction to the user for review and editing before generation begins, and translates the confirmed direction into a canonical naming intent object.

In another aspect, the invention provides a **canonical naming intent representation** — a versioned, normalized intermediate data structure that maps qualitative user preferences to named dimensions (naming styles, sound qualities, emotional qualities, cultural contexts, familiarity preference, distinctiveness preference, temporal preference, and priority weights) independent of the specific intake form version or naming domain (vertical) used to collect them.

In another aspect, the invention provides a **multi-provider AI generation router** that distributes naming generation requests to multiple AI providers, evaluates candidate results using vertical-specific quality adapters and scoring functions, and selects the highest-scoring candidates for presentation to the user.

In another aspect, the invention provides a **reaction-driven taste profile construction system** that derives a structured taste profile from user reactions (positive or negative) to previously presented name candidates, capturing liked and disliked phonetic patterns, stylistic territories, and semantic rationales, and uses that profile to guide subsequent generation rounds toward results more aligned with the user's demonstrated preferences.

In another aspect, the invention provides a **multi-round refinement journey** in which name generation proceeds through sequential rounds, with each round producing a decreasing number of increasingly refined candidates, until the user arrives at a shortlist of finalist names suited for final selection.

In another aspect, the invention provides a **session chain architecture** in which each generation round is stored as a distinct session record linked to the prior round through a parent session reference, enabling replay of briefs and results, reconstruction of taste profiles from any point in the chain, enforcement of round completion requirements, and exclusion of previously presented names from later rounds.

In another aspect, the invention provides **priority weights propagation** in which a user's named priority weights are stored in the canonical naming intent object and propagated into both the vertical-specific quality adapter scoring function and the taste thesis builder used to construct AI prompts, so that otherwise identical

preference inputs may produce different scoring emphasis and different prompt emphasis based on what the user identifies as most important.

In another aspect, the invention provides a **versioned intake schema migration system** that stores field definitions, allowed values, deprecated aliases, and migration rules for each intake schema version, and automatically translates stored intake data to the current schema version when read, preserving session utility without requiring re-collection of user input.

In another aspect, the invention provides a **multi-vertical architecture** in which a single shared engine infrastructure serves multiple distinct naming domains — including at minimum baby names, pet names, and business names — through configuration-driven vertical definitions that specify domain-specific intake questions, AI prompt context, quality scoring dimensions and weights, and visual presentation, while preserving identical underlying platform behavior across all verticals.

In another aspect, the invention provides **Advanced Taste State architecture with confidence modeling, uncertainty tracking, contradiction resolution, and exploration planning**. This aspect maintains a persistent machine-readable preference model across a session chain and actively selects future generation probes to resolve uncertainty and discover additional preference territory.

BRIEF DESCRIPTION OF THE DRAWINGS

The following figures illustrate preferred embodiments of the invention. They are described for reference; formal drawings will be included in the non-provisional application.

Figure 1 — System architecture overview showing the intake pipeline, canonical intent layer, model router, quality framework, storage layer, and refinement orchestrator.

Figure 2 — Sequential intake flow: one-question-at-a-time presentation, optional and required question handling, direction review screen, edit-from-review navigation, and confirmation before generation.

Figure 3 — Canonical Naming Intent data structure showing fields: vertical, naming_target, gender_context, cultural_contexts, naming_styles, familiarity_preference, distinctiveness_preference, sound_qualities, emotional_qualities, priority_weights, and extensions.

Figure 4 — Multi-provider routing flow: parallel or sequential provider dispatch, provider result collection, quality adapter scoring per candidate, candidate selection by score, fallback on shortfall.

Figure 5 — Quality adapter architecture: vertical-specific score dimensions, dimension weights summing to 1.0, model score keys, taste thesis builder, and explanation improvement hooks.

Figure 6 — Taste profile construction from session reaction chain: phonetic signal extraction, territory and rationale aggregation, style scoring by tag, rejected lane tracking, and profile serialization.

Figure 7 — Multi-round refinement journey: Round 1 (broad set, e.g. 8 names), reaction capture, taste profile build, Round 2 (refined set using taste profile), Round 3 (finalist shortlist), compare/choose flow.

Figure 8 — Multi-vertical configuration schema: slug, display_name, intake_questions, prompt_context, quality adapter registration, and vertical-specific visual configuration.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

1. Overview

The NamEngine platform is a web-delivered, server-side application that operates as an intelligent naming engine. It accepts user input through a structured guided intake, processes that input through a canonical normalization layer, generates name candidates using one or more AI language model providers, scores and ranks candidates using domain-specific quality criteria, presents candidates to the user for reaction, builds a taste profile from those reactions, and uses the taste profile to guide one or more subsequent generation rounds.

The platform is designed as a multi-vertical system. A **vertical** is a naming domain configuration that binds together: a set of intake questions, a prompt context string supplied to the AI, a set of quality scoring dimensions and weights (the quality adapter), and a visual presentation configuration. The same platform engine serves all verticals without modification to core logic.

FIG. 1

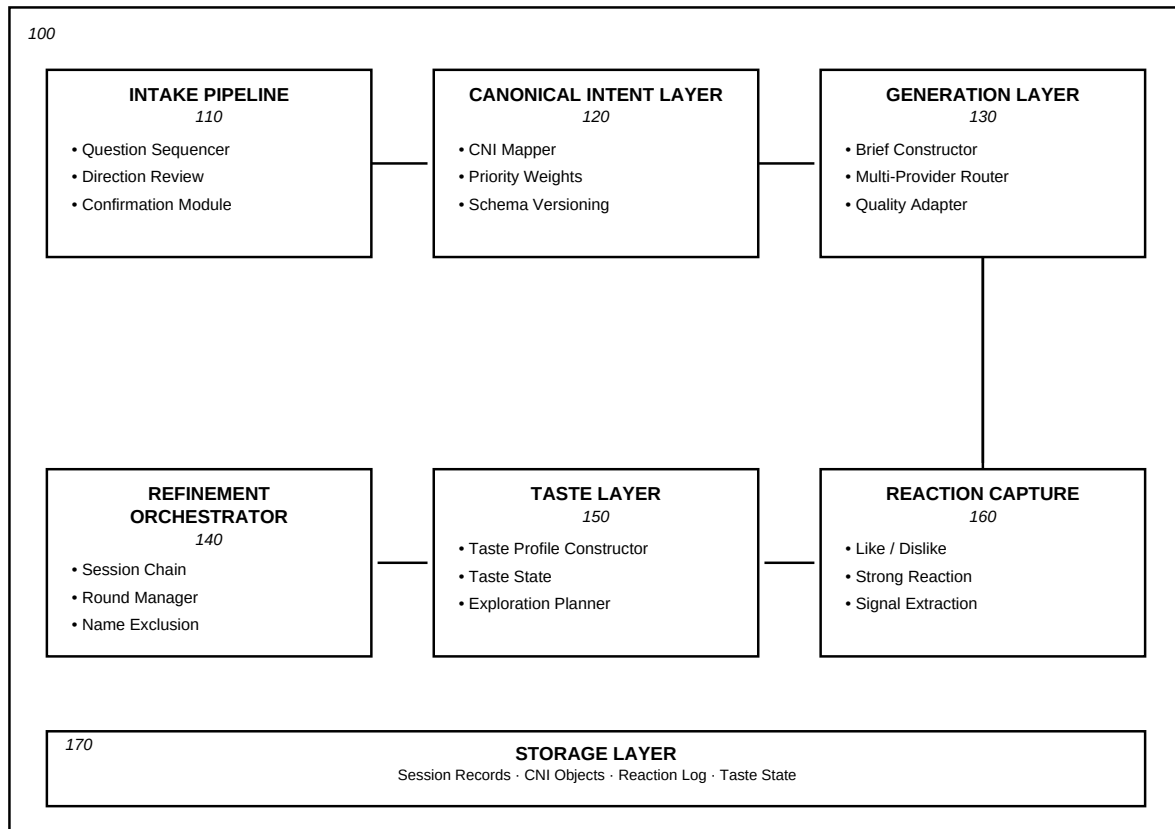


FIG. 1

In the preferred embodiment, three verticals are initially deployed:

- **Baby** — for naming a newborn or expected child
- **Pet** — for naming a companion animal
- **Business** — for naming a commercial enterprise, product, or brand

The invention is not limited to these three domains. Additional verticals (e.g., characters, products, places, services) may be defined and registered using the same configuration framework.

2. Structured Sequential Preference Elicitation

2.1 Intake Question Structure

Each vertical defines a set of intake questions encoded as structured data objects. Each question specifies:

- A unique identifier
- A display label

- A question kind (text, choice, multi-select)
- Whether the question is required or optional
- An ordered set of allowed choice values (for choice questions)
- Optional help text
- Optional placeholder text for text fields

Questions are stored in the vertical configuration and are versioned. Changes to question definitions are managed through an intake migration system that translates older stored answers to newer schema versions.

2.2 One-Question-at-a-Time Presentation

Rather than presenting all intake questions simultaneously in a form, the system presents one question at a time in a sequential guided flow. This design has several functional effects:

- It reduces cognitive load and encourages thoughtful responses
- It allows for auto-advance behavior (automatically advancing to the next question when a choice is selected in a single-select question)
- It enables per-question contextual encouragement copy without form-level noise
- It creates a conversational rhythm that treats naming as a reflective process rather than a data entry task

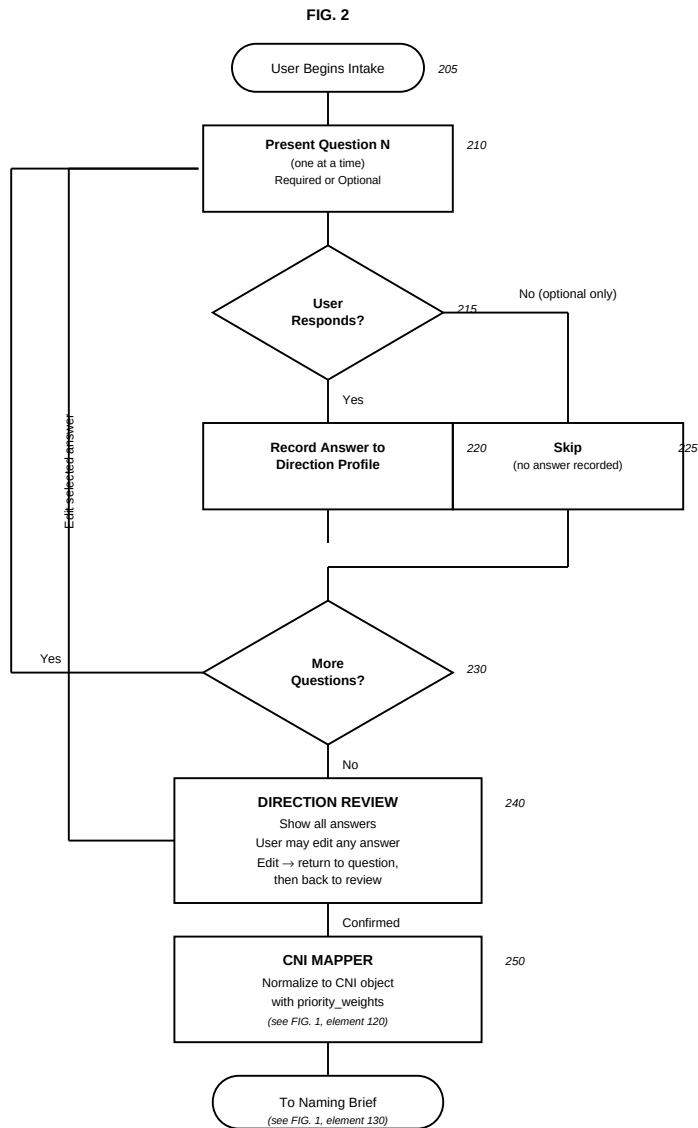


FIG. 2

Navigation between questions is provided via Back and Next controls. On optional questions, advancing via Next without providing an answer is permitted and recorded as a non-response (the answer is omitted from the naming brief). Skip controls are explicitly absent from the interface; the Next control on an optional question serves this function silently.

A progress indicator is displayed throughout the intake showing the user's position in the question sequence.

2.3 Direction Review Before Generation

After all intake questions are presented and answered (or skipped), the system presents a **direction review screen** that displays all collected answers in a unified summary view before any name generation occurs. This screen:

- Shows each question label and the user's answer (or a neutral indicator if unanswered)
- Provides an edit control for each answer
- Activating the edit control for a given answer returns the user to that question in the intake flow
- After editing, the user is returned to the direction review screen rather than continuing through subsequent questions
- The user confirms their direction from the review screen, which triggers name generation

The direction review serves as a quality gate that ensures the user's expressed preferences are accurate before committing to AI generation. It also reinforces the user's sense of authorship over the resulting names.

2.4 Intake Field Validation and Normalization

The system validates all intake field values against the schema defined for each field. Validation includes:

- Type checking (string, integer, boolean, string_list, object)
- Allowed value enforcement for choice fields
- Length and item count bounds
- Normalization (e.g., trimming whitespace, converting aliases to canonical field names)
- Applicable-when conditions (fields may be conditionally applicable based on other field values)

A versioned intake schema system tracks field definitions, aliases, deprecated aliases, and migration rules. When stored intake data from a prior schema version is loaded, it is automatically migrated to the current version.

3. Canonical Naming Intent

3.1 Purpose and Design

The **Canonical Naming Intent** (CNI) is a versioned, normalized intermediate representation that bridges the raw intake responses and the AI generation layer. Its purpose is to:

- Decouple the intake form schema from the generation prompt schema
- Allow intake forms to evolve (new questions, new options, renamed fields) without breaking the generation pipeline
- Provide a stable serialization format that can be stored, compared, and diffed across sessions
- Enable intent-based routing and quality scoring independent of intake version

FIG. 3300

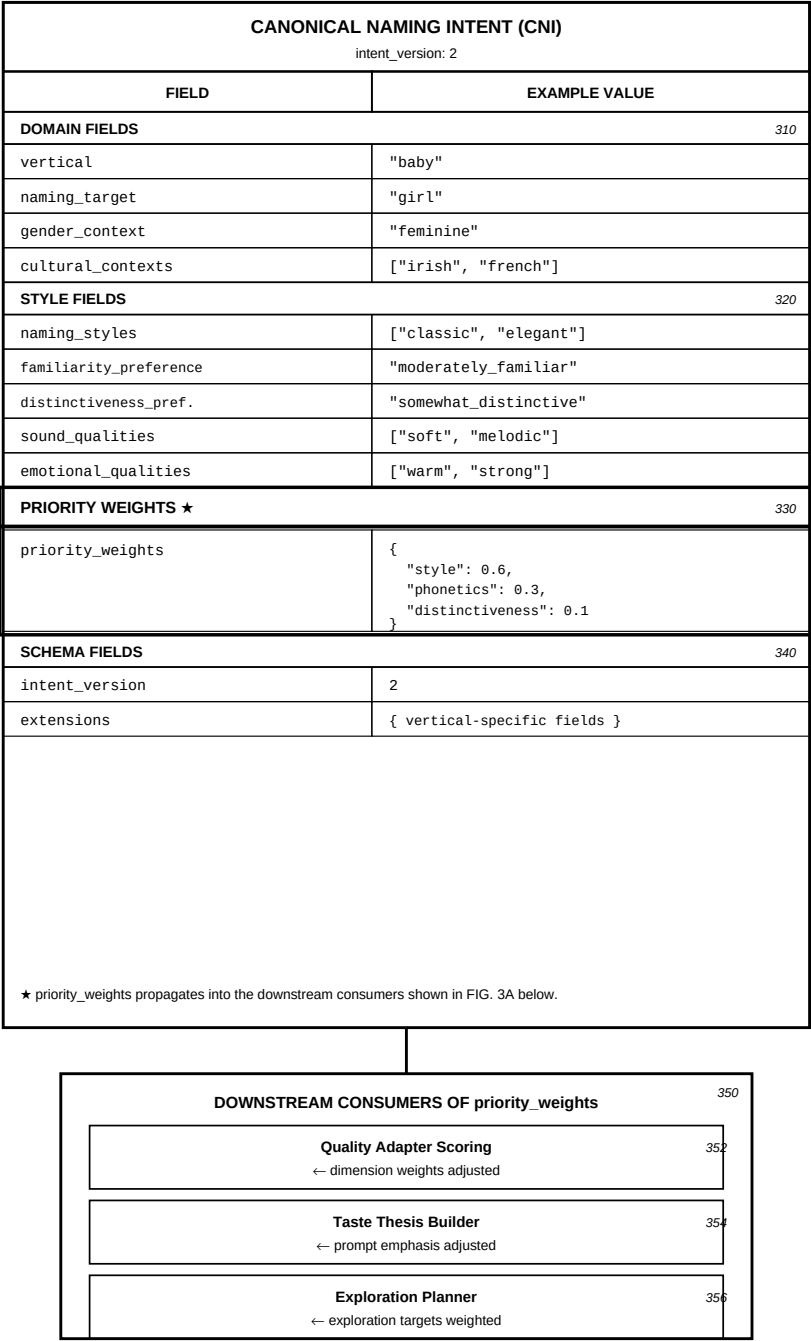


FIG. 3

3.2 CNI Data Structure

The CNI is an immutable frozen data object containing the following named fields:

Field	Description
vertical	Lowercase slug identifying the naming domain
naming_target	The entity being named (e.g., "baby boy," "dog," "business")
gender_context	Expressed gender preference or constraint
cultural_contexts	Cultural, ethnic, or linguistic backgrounds relevant to naming
naming_styles	Named aesthetic style preferences (e.g., "classic," "modern," "nature-inspired")
familiarity_preference	Preference for familiar versus uncommon names
distinctiveness_preference	Preference for distinctive versus blending-in names
sound_qualities	Expressed preferences for phonetic characteristics
emotional_qualities	Emotional resonance preferences (e.g., "warm," "strong," "playful")
strength_softness	Relative preference on a strength-to-softness spectrum
temporal_preference	Preference for names that feel timeless, classic, vintage, or current
discovery_preference	Preference for well-known names versus rare discoveries
family_personal_context	Family naming traditions, honor names, or personal constraints
honor_name_influence	Specific names to honor or draw inspiration from
usage_contexts	Contexts in which the name will be used (e.g., formal, casual, professional)
professional_usability	For business names: professional context and register
geographic_language_contexts	Geographic or language region contexts
avoidances	Explicit constraints: name types, sounds, or associations to avoid
notes	Free-form additional context
priority_weights	Named weight map indicating which dimensions matter most to this user
source_intake_version	The intake schema version from which this intent was derived
normalization_version	The normalization function version applied
intent_version	The CNI schema version identifier
extensions	Vertical-specific extensions for domain-specific fields not covered by the core schema

The `priority_weights` field is a key innovation: it allows the user's expressed priorities to propagate into the quality scoring and prompt construction layers, dynamically adjusting which dimensions of name quality matter most for this specific user's request.

3.2a Worked Example: *priority_weights* Propagation

The following example illustrates how two users with identical base preferences but different priority weights receive differently scored and differently prompted generation results.

User A completes intake indicating: classic style, moderate distinctiveness, soft phonetics. Priority weights: { "style": 0.6, "phonetics": 0.3, "distinctiveness": 0.1 }

User B completes identical intake responses. Priority weights: { "style": 0.2, "phonetics": 0.2, "distinctiveness": 0.6 }

The CNI objects for both users share the same dimension values but carry different `priority_weights` maps. When the quality adapter scores a candidate name, it applies the weights as dimension multipliers:

- For User A: a name scoring high on style and phonetics (e.g., "Eleanor") receives a high composite score even if it is not highly distinctive. A highly distinctive but stylistically neutral name (e.g., "Zephyrine") scores lower.
- For User B: the same "Eleanor" scores lower because distinctiveness is underweighted in the composite. "Zephyrine" scores higher because distinctiveness carries 0.6 weight.

Additionally, the taste thesis builder uses `priority_weights` to adjust the emphasis directives in the AI generation prompt:

- User A's prompt: *"Prioritize classic, time-honored style. Soft phonetics are important. Distinctiveness is a lower priority."*
- User B's prompt: *"Prioritize names that stand out and feel memorable and rare. Style and phonetics are secondary."*

The result is that two users with identical stated preferences receive meaningfully different candidate sets, scored and generated according to their individually expressed priorities. This propagation occurs without any manual prompt engineering by the system operator.

3.3 CNI Serialization and Versioning

The CNI is serializable to a compact JSON string using sorted keys for deterministic output. This serialization is used for storage, session comparison, and change detection. The `intent_version` field identifies the schema version, enabling forward compatibility as the CNI schema evolves.

4. Naming Brief Construction

The **NamingBrief** is the object passed to the generation layer. It contains:

- The vertical slug
- The raw intake inputs (key-value map)
- Liked example names from prior rounds (populated during refinement)
- Rejected example names from prior rounds
- Avoidance instructions
- Free-form notes
- The serialized Canonical Naming Intent
- Intake metadata (schema version, normalization version)

The naming brief is persisted with each session record, enabling reproducibility and supporting the intake migration system.

5. Multi-Provider AI Generation Router

5.1 Provider Architecture

The **model router** distributes name generation requests to one or more AI language model providers. In the preferred embodiment, supported providers include OpenAI, Anthropic Claude, Google Gemini, and Groq. A deterministic **fallback provider** is also available that draws from curated name pools for situations where all AI providers fail or are unavailable.

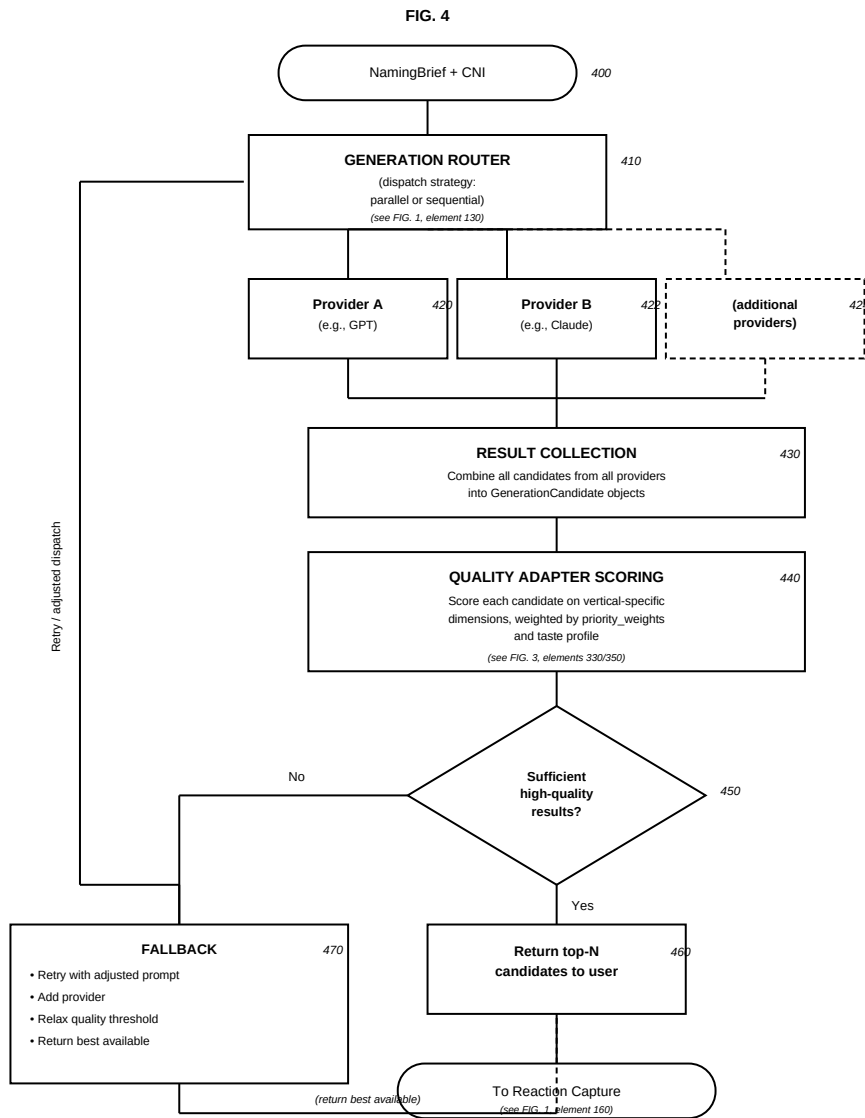


FIG. 4

For each generation request, the router:

Selects which providers to call, based on configuration and prior performance data

Dispatches generation requests to each selected provider

- Collects provider results, each of which is a set of `NameResult` objects
- Passes all provider results through the quality scoring pipeline
- Selects the highest-scoring candidates for presentation

5.2 Provider Results and Candidate Scoring

Each provider invocation produces a `ProviderResult` containing:

- The provider identifier
- A list of `NameResult` objects
- Whether the generation succeeded or failed

Each `NameResult` contains:

- The name string
- A phonetic pronunciation guide
- A plain-language explanation of why the name fits
- A set of named quality scores (model-assessed dimensions specific to the vertical)
- A set of tags (stylistic and semantic categories)
- Validation results

The router wraps each `NameResult` in a `GenerationCandidate` object that includes the provider origin and enables cross-provider comparison.

5.3 Quality Adapter Scoring

Each vertical has a registered **QualityAdapter** that defines:

- A set of named scoring dimensions relevant to that domain
- Weights for each dimension that sum to 1.0
- A `score_dimensions` function that computes a score vector for each candidate
- A `build_taste_thesis` function that converts a `NamingBrief` and taste profile into a natural-language guidance string for the AI prompt
- An optional `improve_explanations` function that post-processes AI-generated explanations for consistency
- An optional `evaluate_attributes` function for additional domain-specific attribute scoring

The quality framework applies each adapter's `score_dimensions` function to every candidate and produces a composite quality score used to rank and select candidates. Candidates from lower-quality providers or with lower quality scores are deprioritized or excluded.

FIG. 5

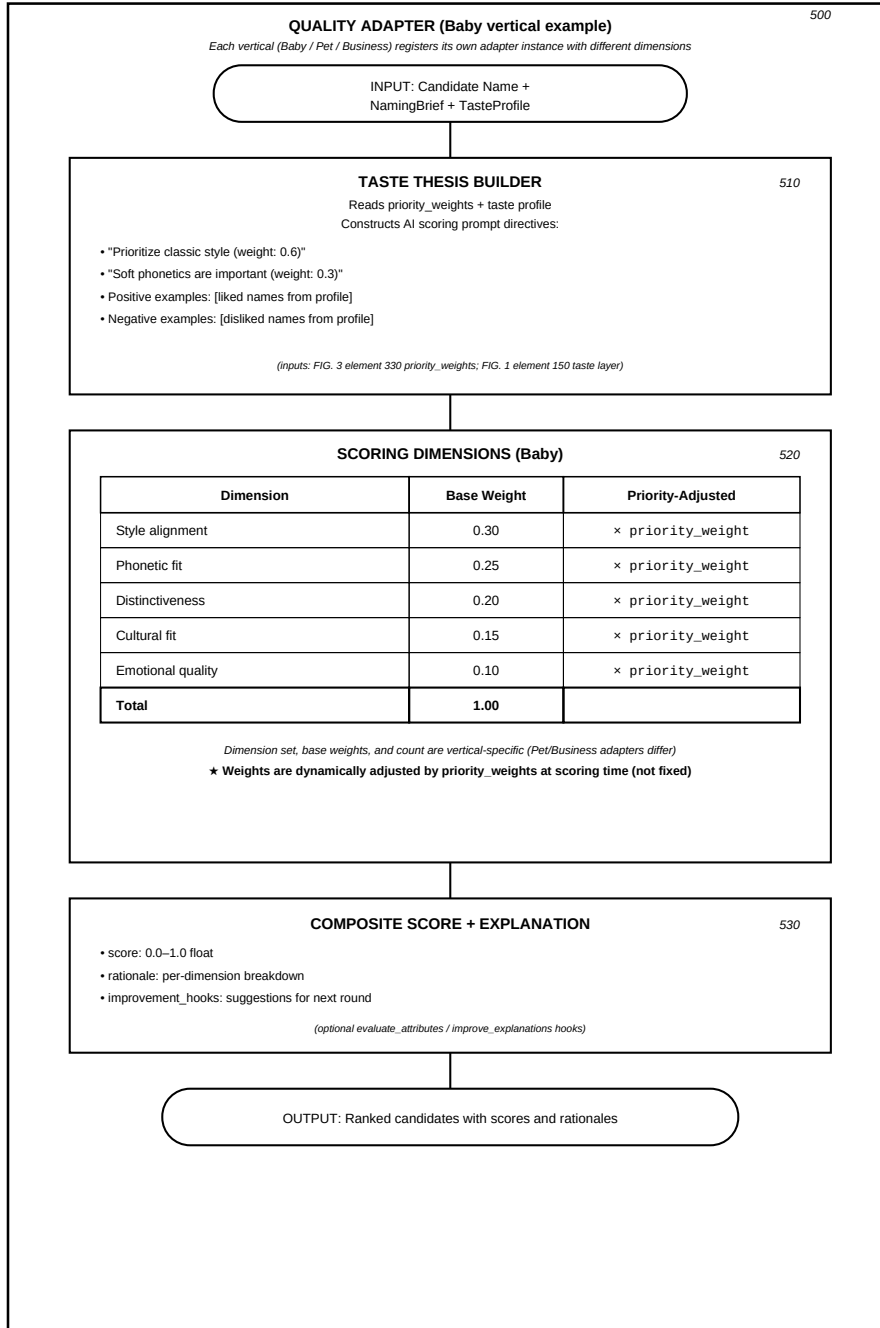


FIG. 5

Example quality dimensions for the baby vertical include: phonetic appeal, stylistic alignment with expressed preferences, cultural appropriateness, emotional resonance, and rarity calibration. Example dimensions for the business vertical include: memorability, pronounceability, spelling clarity, brandability, and domain/trademark availability signal.

5.4 Fallback on Shortfall

If the total number of passing candidates from AI providers falls below the target count for a round, the router optionally invokes the fallback provider to fill the shortfall. This ensures users always receive a complete set of name candidates regardless of AI provider availability.

6. Reaction Capture and Taste Profile Construction

6.1 Reaction Model

After each generation round, the user reviews the presented name candidates and assigns a reaction to each. In the preferred embodiment, two active reaction values are supported:

- **Love** — the user strongly likes this name
- **No** — the user does not want this name

(A third historical value, "Maybe," is retained for data compatibility but is not exposed in the current product interface.)

Reactions are stored in association with the session record and the specific result records they reference.

6.2 Taste Profile Construction

After reactions are collected, the system constructs a **TasteProfile** by analyzing the full reaction history across all prior rounds in the session chain. The construction process:

Loads all session snapshots in the chain (round 1 through current)

For each reaction, retrieves the corresponding `NameResult` metadata

For loved names: extracts phonetic signals, style tags, scoring dimensions, and semantic territories; increments liked-sound, liked-territory, liked-rationale, and style-score counters

For rejected names: extracts the same signals; increments disliked-sound, disliked-territory, disliked-rationale, and rejected-lane counters

Aggregates counters using configurable top-N selection (e.g., top 4 liked sounds)

Produces a `TasteProfile` object containing:

- Lists of loved, maybe, and rejected name strings
- Top liked and disliked phonetic patterns
- Top liked and disliked stylistic territories
- Normalized style preference scores across tagged dimensions
- Rejected lane signals (stylistic territories to avoid)

FIG. 6

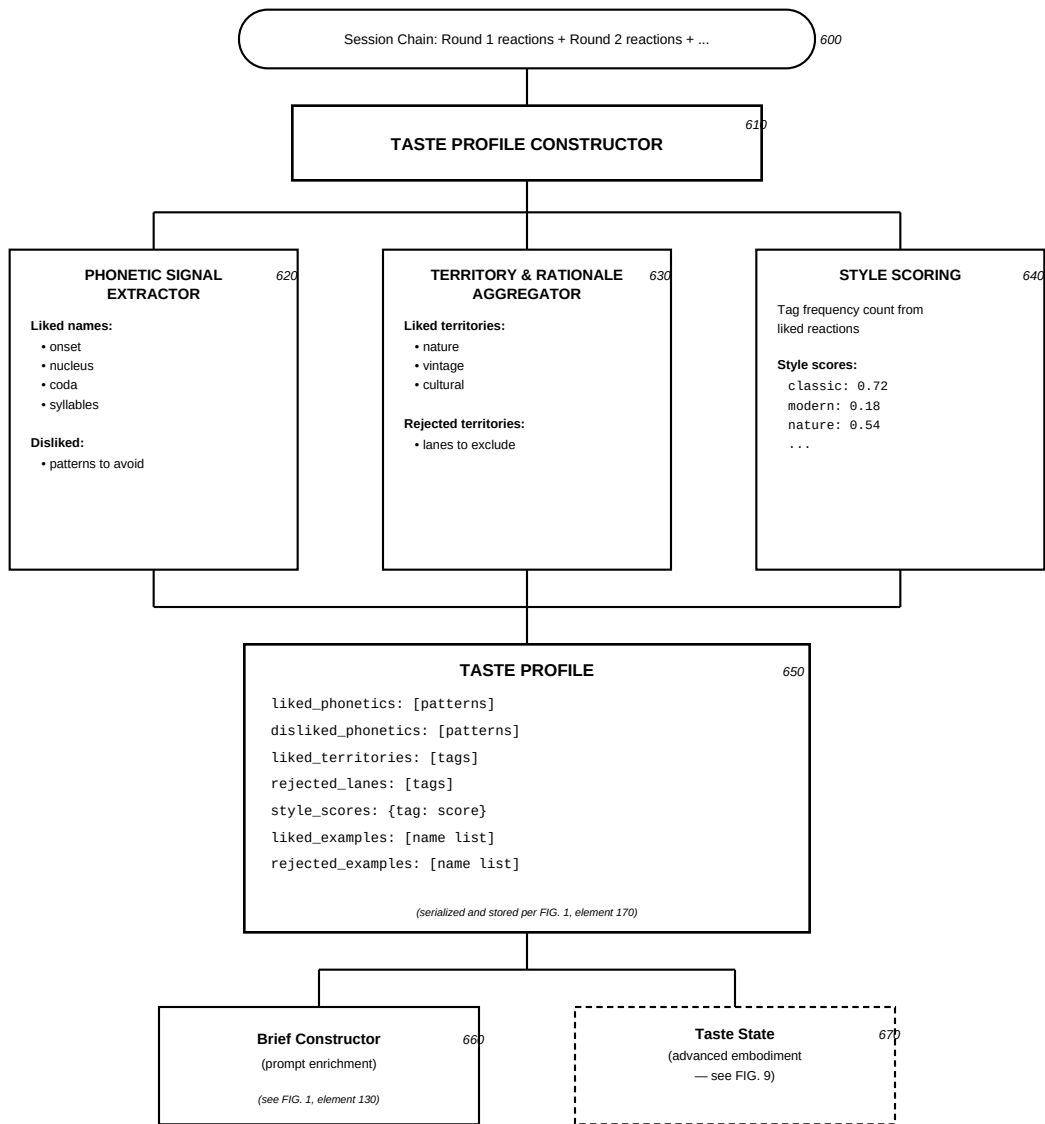


FIG. 6

The TasteProfile is serialized and stored with the session. It is supplied to the model router as input for subsequent generation rounds.

6.3 Taste Profile Use in Generation

The taste profile is incorporated into subsequent generation rounds in two ways:

Prompt augmentation: The vertical's `build_taste_thesis` function converts the taste profile into a natural-language summary included in the AI prompt. This summary communicates which name types the user responded positively to, which phonetic patterns they preferred, and which territories or styles to avoid.

Brief enrichment: The liked and rejected name lists from the taste profile are added to the `NamingBrief` as `liked_examples` and `rejected_examples`, providing the AI with concrete positive and negative reference points.

7. Multi-Round Refinement Journey

7.1 Round Structure

The naming journey proceeds through a structured sequence of rounds:

- **Round 1:** The system generates a broad set of name candidates (typically 8) based on the `NamingBrief` derived from the user's intake. This round explores the possibility space and collects the user's initial taste reactions.
- **Round 2:** The system generates a refined set of candidates (typically 8) using the taste profile built from Round 1 reactions. Candidates are more targeted to the user's demonstrated preferences. Previously presented names are excluded.
- **Round 3:** The system generates a finalist shortlist (typically 5–6 candidates) using the accumulated taste profile from Rounds 1 and 2. These candidates represent the highest-confidence fit with the user's preferences. The user is nudged toward comparison and final selection.
- **Rounds 4+:** Additional refinement rounds are supported for users who want to continue exploring before committing.

FIG. 7

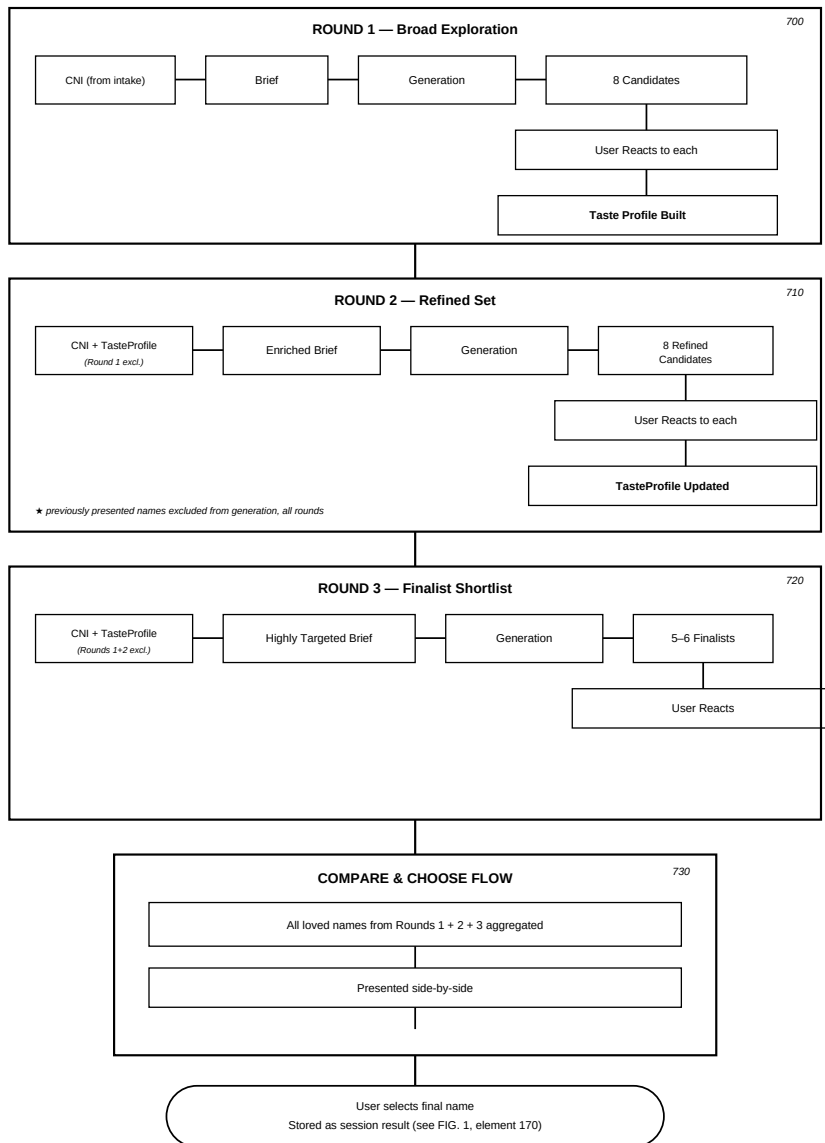


FIG. 7

7.2 Session Chain Architecture

Each round creates a new **session record** linked to the prior round's session via a parent session reference. This chain structure:

- Preserves the full history of names shown, reactions given, and briefs used at each round
- Enables taste profile construction across all prior rounds
- Supports session replay, audit, and comparison
- Allows the system to enforce round completion requirements (e.g., a minimum number of results before advancing)

7.3 Round Transition Logic

When transitioning from one round to the next:

The current session's reactions are validated

The taste profile is rebuilt from the full session chain

A new NamingBrief is constructed by enriching the original brief with taste profile signals

A new session record is created with the incremented round number and parent session reference

The model router generates candidates for the new round, excluding all names from prior rounds

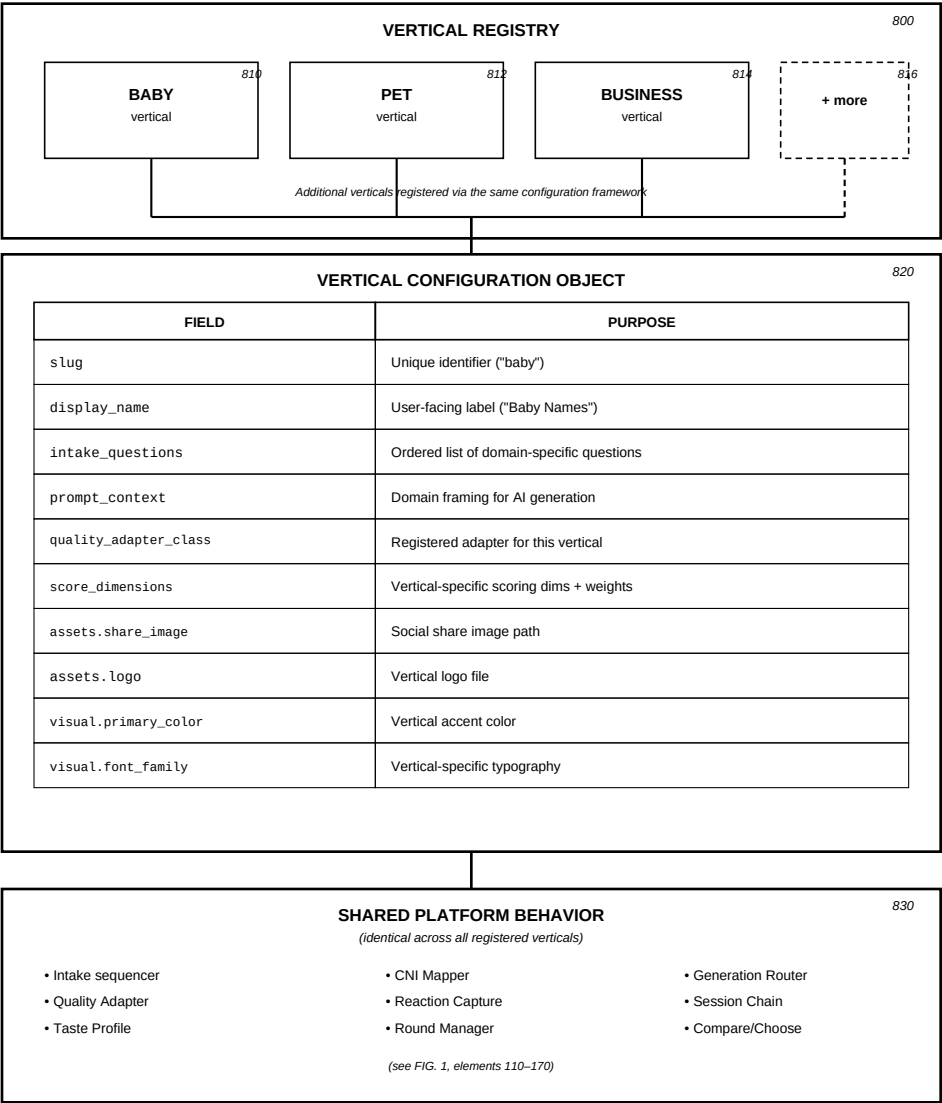
8. Multi-Vertical Platform Architecture

8.1 Vertical Configuration

Each naming domain (vertical) is defined by a `VerticalConfig` object specifying:

- `slug`: A unique lowercase identifier (e.g., "baby," "pet," "business")
- `display_name`: The human-readable vertical name
- `object_label`: The label for the entity being named (e.g., "baby," "pet," "brand")
- `route_prefix`: The URL prefix for this vertical's routes
- `intake_questions`: An ordered tuple of `Question` objects
- `prompt_context`: A domain-specific context string included in AI prompts
- `result_field_labels`: Display labels for result metadata fields
- `validation_modules`: Names of validation functions to apply to generated results
- `theme`: Color and style configuration for visual rendering
- `assets`: Paths to static assets (e.g., logo SVG files)
- `visual`: A structured visual configuration object for the vertical's graphical identity
- `default_result_count`: The default number of names to generate per round

FIG. 8



★ Core principle: platform functionality is identical across verticals;
only configuration (questions, prompts, scoring, visuals) varies

FIG. 8

8.2 Vertical Registration

Vertical configurations are registered at application startup. The platform engine discovers registered verticals and routes incoming requests to the appropriate vertical configuration based on the URL route

prefix. This architecture allows new verticals to be added without changes to the core platform logic.

8.3 Quality Adapter Registration

Each vertical's `QualityAdapter` is independently registered against the vertical slug. The quality framework dispatches scoring through the registered adapter for the relevant vertical. This registry-based design ensures that domain-specific scoring logic is always applied and that adding a new vertical requires only implementing and registering its adapter.

8.4 Identical Behavior Across Verticals

A core design principle of the platform is that user-facing functionality — the intake flow, direction review, generation loading state, results presentation, reaction capture, refinement rounds, compare and choose flow — is identical in structure and behavior across all verticals. What varies is: the questions asked, the AI prompt context, the quality scoring dimensions, the visual design (colors, typography, graphical language), and the copy voice. This ensures that engineering improvements to shared components benefit all verticals simultaneously.

9. Additional Features

9.1 Name Validation

Each vertical defines validation modules that are applied to generated name candidates. Validation checks may include:

- Gender appropriateness filtering (e.g., for baby names, filtering against a list of allowed names for the expressed gender)
- Trademark or domain availability checks (e.g., for business names, querying domain registrar APIs to assess domain availability)
- Content filtering to exclude inappropriate or offensive name strings

Validation results are stored with each candidate and surfaced in the quality scoring pipeline.

9.2 Provider Performance Tracking

The system tracks historical performance metrics for each AI provider across verticals and round types. These metrics include success rate, result quality scores, and generation latency. Provider performance data is available for use in future routing decisions (e.g., preferring providers with higher historical quality scores for a given vertical).

9.3 Intake Schema Migration

As the platform evolves, intake question schemas may change. The system includes an intake migration framework that maps stored intake data from prior schema versions to the current version. Migrations are applied on read, ensuring that session records created under older schemas remain fully functional.

9.4 Compare and Choose Flow

After the refinement journey, the platform offers a compare-and-choose flow in which the user's loved names from all rounds are presented side-by-side for final evaluation. The user may select a final chosen name, which is stored as the session's chosen result.

9.5 Share Flow

Users may share individual name results or their chosen name via platform-generated share cards. The share mechanism is integrated with the session storage system.

9.6 Mission Control and Telemetry

An internal administrative interface (Mission Control) provides visibility into session activity, generation quality audits, provider performance, and platform telemetry. This interface is not customer-facing.

10. Advanced Embodiment — Taste State Architecture and Exploration Planner

In an advanced embodiment, the platform extends the TasteProfile described above into a persistent **Taste State** architecture and couples that state to an **Exploration Planner** that actively directs subsequent generation rounds. This embodiment treats the user's naming taste as a machine-readable state model that is updated after each reaction and used not only to exploit demonstrated preferences, but also to deliberately probe uncertain, contradictory, or unexplored areas of the naming space.

10.1 Taste State Data Structure

The Taste State is maintained per user session chain. In one embodiment, a session chain begins with the initial intake-derived generation session and continues through each child refinement session linked by parent session reference. The Taste State may be stored as a serialized object associated with the latest session, reconstructed from the full reaction history, or stored as incremental state snapshots at each round.

The Taste State generalizes the TasteProfile by storing inferred preference dimensions together with confidence, uncertainty, evidence, contradiction, and exploration metadata. Preference dimensions may include, without limitation:

- Phonetic qualities, including opening sounds, ending sounds, vowel shapes, consonant clusters, rhythm, stress pattern, syllable count, softness, sharpness, and pronounceability;
- Naming styles, including classic, modern, vintage, nature-inspired, elegant, playful, minimalist, invented, surname-style, mythological, literary, international, luxury, technical, friendly, and other configured style territories;
- Emotional registers, including warm, strong, calm, joyful, refined, adventurous, whimsical, grounded, premium, trustworthy, energetic, or gentle;
- Distinctiveness level, including familiar, uncommon, rare, bold, conventional, and highly distinctive naming territories;
- Cultural, linguistic, geographic, or regional associations, including associations expressly supplied in intake responses and associations inferred from generated names and user reactions.

For each inferred preference dimension, the Taste State may store one or more of the following fields:

- A confidence level represented as a numeric value, such as a floating point value from 0.0 to 1.0, indicating the system's current confidence that the dimension reflects the user's preference;
- An uncertainty flag indicating that evidence for the dimension is insufficient, contradictory, sparse, or not yet tested;
- Strength-of-evidence counters, including confirming reaction counts and disconfirming reaction counts, optionally separated by reaction strength;
- Positive signal inventories identifying specific phoneme patterns, morpheme structures, syllable counts, stress patterns, style tags, semantic territories, and other markers associated with liked names;
- Negative signal inventories identifying corresponding signals associated with disliked or rejected names;
- Contradictory evidence records identifying pairs or groups of reactions that suggest conflicting preferences, such as liking a name with a phonetic pattern that was previously inferred to be disliked, or rejecting a name in a style territory that was previously inferred to be preferred;
- A rejected lane registry identifying territories that should be avoided, deprioritized, or reintroduced only as deliberate probes;
- An unexplored preference territory map identifying preference dimensions or naming territories that have not yet been meaningfully probed by the system.

The unexplored preference territory map may include configured dimensions for the active vertical, dimensions implied by the intake responses but not yet tested by generated names, dimensions common to the vertical but absent from the user's prior candidate set, and dimensions suggested by contradictory evidence. Each territory may be assigned a priority score based on expected information gain, relevance to the user's intake, vertical-specific importance, and whether resolving the territory would improve generation quality.

10.2 Initial Taste State From Intake and Canonical Naming Intent

Before the first generation round, the system may initialize the Taste State from intake responses and the Canonical Naming Intent. Intake-derived signals can establish tentative preference dimensions with moderate or low confidence. For example, a user's selection of a soft naming style may initialize a preference dimension for soft phonetic qualities, while an expressed desire for an uncommon name may initialize a distinctiveness preference. Because intake responses are declarative rather than behaviorally demonstrated, the initial confidence levels may be lower than confidence levels produced by repeated reactions to actual candidates.

The initial Taste State may also identify unexplored territories that should be tested in early rounds. For instance, where intake indicates that the user wants a name that is both familiar and distinctive, the system may mark adjacent territories such as modern familiar names, rare classic names, and uncommon-but-pronounceable names as candidate exploration territories.

10.3 Exploration Planner

The Exploration Planner is a system component that reads the current Taste State before a generation round and produces an **Exploration Strategy** for that round. The Exploration Planner may operate after intake

normalization and before brief construction, or after a preliminary brief is constructed and before provider dispatch.

The Exploration Planner identifies dimensions with high uncertainty, sparse evidence, contradictory evidence, or high unexplored-territory priority. It then selects exploration targets indicating which preference territories should be probed in the next round. The planner balances exploitation and exploration. Exploitation generates candidates aligned with high-confidence preferences already inferred from the Taste State. Exploration generates candidates designed to resolve uncertainty, test a hypothesis, or probe a territory not yet meaningfully represented in prior candidate sets.

In one embodiment, the Exploration Planner produces a structured Exploration Strategy object containing:

- Target dimensions selected for the next generation round, such as phonetic softness, uncommon classical style, short two-syllable structures, premium business register, or playful pet-name cadence;
- An exploration ratio specifying what fraction of generated names should be exploratory rather than exploitative;
- Specific probe directives describing characteristics that exploratory candidates should contain, avoid, or vary;
- Exploitation directives describing high-confidence preference dimensions that should be preserved;
- Contradiction-resolution directives identifying conflicting signals to test;
- Exclusion directives identifying names or territories already exhausted or rejected.

A probe directive may take the form of a natural-language generation instruction, a structured feature constraint, or both. Examples include: "generate 3 names with soft fricative openings to test preference for that phoneme class"; "include two uncommon-but-recognizable names to distinguish rarity preference from unfamiliarity aversion"; "test whether the user dislikes surname-style business names generally or only harsh consonant endings"; or "probe warm nature-adjacent names without using floral imagery."

The exploration ratio may be fixed by round type or computed dynamically. For example, an early round may use a higher exploration ratio to map the user's taste, while a final round may use a lower exploration ratio to concentrate on high-confidence finalist candidates. If the Taste State includes unresolved contradictions, the planner may temporarily raise the exploration ratio for contradiction-resolution probes.

10.4 Use of Exploration Strategy by Brief Constructor, Generation Router, and Quality Adapter

The Exploration Strategy is consumed by the Brief Constructor and Quality Adapter in lieu of, or in addition to, standard TasteProfile signals. The Brief Constructor may incorporate the Exploration Strategy into the NamingBrief as structured fields, prompt instructions, or both. Exploitative directives may be included as positive guidance, while exploratory directives may be represented as round-specific candidate slots or feature constraints.

The Generation Router may pass the Exploration Strategy to one or more AI providers as part of the provider prompt, or may split the request into exploitative and exploratory sub-requests. For example, if eight names are requested and the exploration ratio is 0.375, the router may request five exploitative candidates aligned with high-confidence preferences and three exploratory candidates designed to test target dimensions.

The Quality Adapter may score candidates against both the Taste State and the Exploration Strategy. Exploitative candidates may be rewarded for close alignment with high-confidence dimensions, while exploratory candidates may be rewarded for satisfying probe directives even when they intentionally depart from some high-confidence preferences. The Quality Adapter may therefore preserve information-gathering candidates that would otherwise be excluded by a purely exploitative ranking. The presented candidate set may include both high-fit names and controlled probes selected for expected learning value.

10.5 Evidence Extractor and Taste State Update

After candidates are presented to the user, the Reaction Capture module records reactions such as like, dislike, strong like, and strong dislike. An Evidence Extractor analyzes each reacted-to name and extracts phonetic signals, style markers, territory markers, syllable patterns, morpheme structures, emotional registers, cultural or geographic associations, semantic tags, and other metadata available from the generated candidate record or computed by deterministic analysis.

For a liked name, the Evidence Extractor extracts the name's signals and updates matching preference dimensions by increasing confirming evidence counters, increasing confidence levels where consistent with prior evidence, and adding the extracted signals to the positive signal inventory. For example, if the user likes multiple two-syllable names with liquid consonant endings, the Taste State may increase confidence that the user prefers that phonetic pattern.

For a disliked name, the Evidence Extractor extracts the same categories of signals and updates the Taste State by increasing disconfirming evidence counters, decreasing confidence in matching positive dimensions where appropriate, adding signals to the negative signal inventory, and adding associated territories to the rejected lane registry. A rejected lane may indicate that future rounds should avoid the lane except when a later Exploration Strategy deliberately reintroduces a controlled probe.

Strong reactions are weighted more heavily than mild reactions. A strong positive reaction, such as a heart or equivalent high-affinity signal, may increase confidence and confirming evidence more than a mild positive reaction. A strong negative reaction, such as an X or equivalent rejection signal, may decrease confidence or increase negative evidence more than a mild negative reaction. The weights may be fixed, vertical-specific, or learned from platform performance data.

If a reaction conflicts with an established preference dimension, the system records contradictory evidence. For example, if the Taste State indicates high confidence that the user dislikes invented names but the user strongly likes an invented candidate, the system may reduce confidence in that dimension, mark the dimension as contradictory, and store a contradiction record linking the prior negative evidence and the new positive evidence. The contradiction record may include the candidate names, reaction values, extracted signals, timestamps or round numbers, and a hypothesis for resolution. Subsequent Exploration Strategies may target the contradiction by generating names that vary one factor at a time, allowing the system to determine whether the prior inference was overbroad.

Taste State updates may also modify the unexplored preference territory map. Territories that have now been probed with sufficient evidence may be marked as explored. Territories implicated by contradictions may be reprioritized for future probing. Territories adjacent to newly liked signals may be added as candidate exploration targets.

10.6 Full Architecture Loop

In this advanced embodiment, the full refinement loop proceeds as follows:

The user completes structured intake.

The Canonical Naming Intent is produced from the intake responses.

An initial Taste State is created from intake signals, the CNI, and vertical-specific default exploration territories.

The Exploration Planner reads the Taste State and produces an Exploration Strategy.

The Brief Constructor creates or enriches a NamingBrief using the CNI, Taste State, and Exploration Strategy.

The Generation Router dispatches the brief and exploration directives to one or more generation providers.

Candidate names are returned by the providers.

The Quality Adapter scores candidates against ordinary vertical-specific quality criteria, the Taste State, and the Exploration Strategy.

Selected candidates are presented to the user.

The Reaction Capture module records user reactions.

The Evidence Extractor extracts signals from reacted-to names.

The Taste State Update mechanism adjusts confidence levels, evidence counters, signal inventories, contradiction records, rejected lanes, and the unexplored preference territory map.

The Exploration Planner reads the updated Taste State before the next round and produces a next Exploration Strategy.

The cycle repeats for subsequent rounds until the user reaches a satisfactory shortlist or final selection.

This loop may be expressed as: Intake → CNI → Taste State (initial, from intake signals) → Exploration Planner → Exploration Strategy → Brief Constructor → Generation Router → Candidate Names → Quality Adapter (scored against Taste State and Exploration Strategy) → Presented to User → Reaction Capture → Evidence Extractor → Taste State Update (confidence adjustments, new signals, resolved contradictions, updated unexplored territory map) → Exploration Planner (next round) → repeat.

10.6a Architecture Diagram: Taste Engine Feedback Loop

The following text diagram illustrates the Taste Engine feedback loop across two generation rounds:

Timeline diagram showing the sequence of events:

- ROUND 1
- [Intake]
- [CNI Mapper]
- [Taste State: initial]
- [Exploration Planner]
- [Taste State: initial]

■ ■ ■

- Each round's Taste State is a superset of the prior round's state (monotonically richer)
- The Exploration Planner shifts strategy each round based on what was learned
- Previously presented names are excluded from all subsequent rounds via session chain linkage
- Strong reactions (heart/X) update Taste State with higher evidence weight than mild reactions
- Contradictory evidence is stored explicitly and used to reduce confidence rather than being averaged away

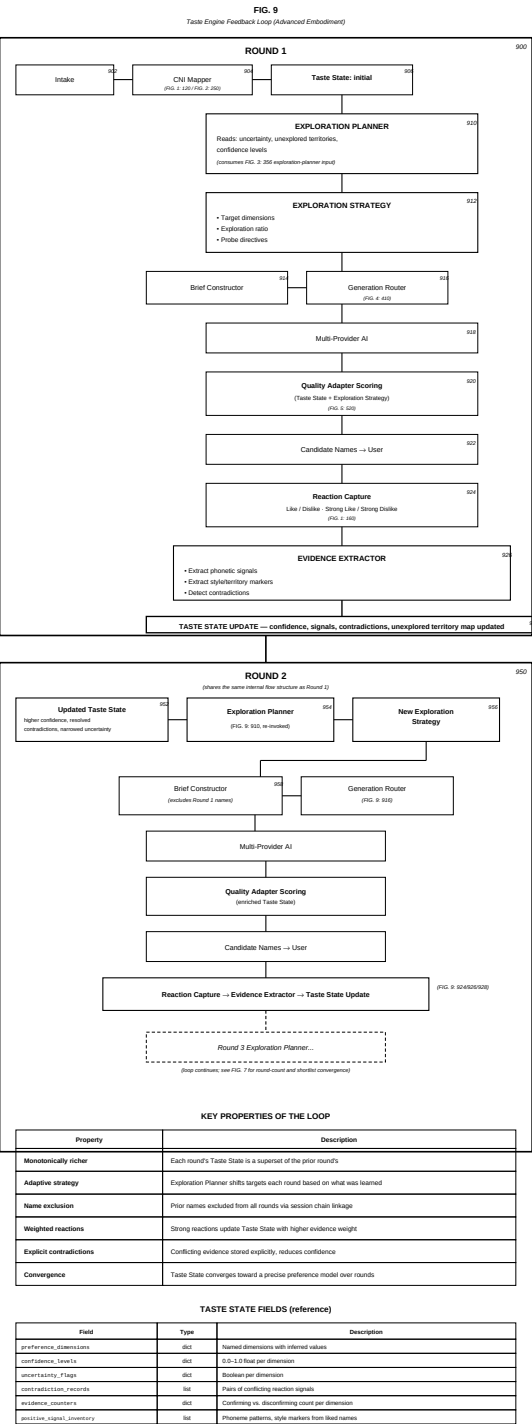


FIG. 9

10.7 Relationship to TasteProfile

The Taste State is an evolution and generalization of the TasteProfile described in Section 6. Existing implementations using the TasteProfile are a subset of the Taste State architecture. The TasteProfile

aggregates reaction-derived liked and disliked patterns for reuse in subsequent rounds. The Taste State extends that reactive aggregation by adding confidence modeling, uncertainty tracking, strength-of-evidence counters, contradiction resolution, positive and negative signal inventories, rejected lane management, unexplored territory mapping, and active exploration planning. Implementations may therefore begin with the simpler TasteProfile embodiment and later add Taste State fields and Exploration Planner behavior without changing the overall session-chain refinement architecture.

INDICATIVE CLAIMS

Note: Formal claims are not required for a provisional patent application. The following indicative claims are provided to frame the scope of the invention for reference when drafting the non-provisional application.

Claim 1. A computer-implemented system for generating personalized name candidates using a combination of canonical intent normalization, vertical-specific quality scoring, and reaction-derived refinement, comprising: a structured sequential intake module configured to present intake questions to a user one at a time, collect user responses, and produce a direction profile; a direction review module configured to present the direction profile to the user for review and editing prior to name generation; a canonical intent mapper configured to normalize the confirmed direction profile into a Canonical Naming Intent object comprising named dimensions including at least naming styles, sound qualities, emotional qualities, familiarity preference, distinctiveness preference, and priority weights; a brief constructor configured to produce a naming brief incorporating the Canonical Naming Intent object; a multi-provider AI generation router configured to dispatch the naming brief to one or more AI language model providers and collect candidate name results from each provider; a vertical-specific quality adapter configured to score and rank the candidate name results using scoring dimensions and weights specific to an active naming vertical; a reaction capture module configured to receive user reactions to the candidate name results; a taste profile constructor configured to derive a machine-readable taste profile from the user reactions, the taste profile comprising at least liked phonetic patterns, disliked phonetic patterns, liked stylistic territories, disliked stylistic territories, and normalized style preference scores; and a multi-round refinement orchestrator configured to use the taste profile in at least one subsequent generation round by enriching the naming brief, adjusting generation guidance, and excluding previously presented names.

Claim 2. The system of claim 1, wherein the taste profile constructor analyzes reactions across a plurality of session records in a session chain and associates each reaction with stored candidate metadata comprising phonetic signals, style tags, scoring dimensions, and semantic rationales.

Claim 3. The system of claim 1, wherein the multi-round refinement orchestrator is configured to: generate a first round of name candidates from the naming brief; collect reactions to the first round; construct a taste profile from the reactions; enrich the naming brief with the taste profile; and generate a subsequent round of name candidates from the enriched naming brief, wherein previously presented names are excluded from subsequent rounds.

Claim 4. The system of claim 1, wherein the multi-provider AI generation router is further configured to: dispatch generation requests to a plurality of AI language model providers; aggregate candidate results from

all providers; apply the vertical-specific quality adapter scoring function to each candidate; and select candidates by composite quality score, falling back to a deterministic name pool provider when the number of passing candidates falls below a target threshold.

Claim 5. The system of claim 1, wherein the vertical-specific quality adapter comprises: a set of named scoring dimensions specific to a naming domain; dimension weights that sum to 1.0; a scoring function that computes a score vector for each candidate name result; and a taste thesis builder that converts the naming brief and the taste profile into a natural-language guidance string for inclusion in AI generation prompts.

Claim 6. The system of claim 1, further comprising a multi-vertical configuration registry in which a plurality of naming domain configurations are independently registered, each naming domain configuration specifying at least: an ordered set of intake questions, a domain-specific AI prompt context string, a registered quality adapter, and a visual presentation configuration, wherein the platform engine applies identical underlying behavior across all registered naming domains.

Claim 7. A computer-implemented method for generating personalized name candidates using a combination of canonical intent normalization, vertical-specific quality scoring, and reaction-derived taste refinement, comprising: presenting intake questions to a user sequentially, one question at a time, and collecting responses; presenting a direction review to the user showing all collected responses before generating any name candidates; receiving user confirmation or edits of the direction review; normalizing the confirmed responses into a Canonical Naming Intent object comprising named dimensions including at least naming styles, sound qualities, emotional qualities, familiarity preference, distinctiveness preference, and priority weights; constructing a naming brief from the Canonical Naming Intent object; routing the naming brief to one or more AI language model providers; scoring candidate name results from each provider using a vertical-specific quality adapter comprising domain-specific scoring dimensions and weights; selecting highest-scoring candidates; presenting the selected candidates to the user; collecting user reactions to presented candidates; constructing a machine-readable taste profile from the collected reactions; enriching a subsequent naming brief using the taste profile; and using the enriched subsequent naming brief to guide at least one subsequent generation round.

Claim 8. The method of claim 7, wherein normalizing the confirmed responses comprises mapping intake field values to named canonical dimensions including at least: naming styles, sound qualities, emotional qualities, cultural contexts, familiarity preference, distinctiveness preference, and a priority weights map that specifies relative importance of dimensions for this user.

Claim 9. The method of claim 7, wherein constructing the taste profile comprises: loading all session records in a session chain; extracting phonetic signals from names associated with positive reactions; extracting phonetic signals from names associated with negative reactions; aggregating style tag scores weighted by reaction value; and producing liked and disliked phonetic pattern lists, territory lists, and style score vectors for use in subsequent generation prompts.

Claim 10. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to: receive a naming domain configuration comprising an intake question set, a domain-specific prompt context, and a quality adapter registration; present intake questions one at a time to a user; collect and normalize user responses into a Canonical Naming Intent object; present a direction review of collected responses to the user before generation; generate name candidates by routing

a naming brief incorporating the Canonical Naming Intent object to a plurality of AI providers; score and rank candidates using the registered vertical-specific quality adapter; present top-ranked candidates to the user; collect user reactions; construct a machine-readable taste profile from reactions; and generate at least one subsequent round of refined name candidates using the taste profile, wherein the sequence of steps is applied identically regardless of which naming domain configuration is active.

Claim 11. A computer-implemented system comprising: a canonical intent mapper configured to normalize confirmed naming preference inputs into a Canonical Naming Intent object; a vertical-specific quality adapter configured to score generated candidate names using scoring dimensions and weights specific to an active naming vertical; a taste profile constructor configured to construct and use a reaction-derived taste profile in subsequent generation rounds; and a session chain architecture wherein each generation round creates a new session record linked to the prior round via a parent session reference, enabling: full replay of any round's brief and results; taste profile reconstruction from any point in the chain; enforcement of round completion requirements; and exclusion of all previously presented names from subsequent rounds.

Claim 12. The system of claim 1, wherein the Canonical Naming Intent object includes a `priority_weights` field comprising a named weight map specified by the user during intake, and wherein the `priority_weights` field is used to: dynamically adjust the dimension weights applied by the quality adapter scoring function; and adjust the emphasis of named dimensions in the AI prompt constructed by the taste thesis builder, such that two users with identical preference inputs but different priority weights receive differently scored and differently prompted generation requests.

Claim 13. A computer-implemented system comprising: a canonical intent mapper configured to normalize confirmed naming preference inputs into a Canonical Naming Intent object; a vertical-specific quality adapter configured to score generated candidate names using scoring dimensions and weights specific to an active naming vertical; a taste profile constructor configured to construct and use a reaction-derived taste profile in subsequent generation rounds; a versioned intake schema registry storing field definitions, allowed values, deprecated aliases, and migration rules for each schema version; and an intake migration module that, on reading stored session intake data, detects the stored schema version, applies all applicable migration rules to translate field names and values to the current schema version, and produces a current-version intake object without requiring re-collection of user input.

Claim 14. A computer-implemented method comprising: normalizing confirmed naming preference inputs into a Canonical Naming Intent object; generating candidate names using a naming brief incorporating the Canonical Naming Intent object; scoring candidate names using a vertical-specific quality adapter comprising scoring dimensions and weights specific to an active naming vertical; collecting user reactions across a plurality of generation rounds in a session chain; constructing and using a reaction-derived taste profile in subsequent generation rounds; aggregating all name candidates associated with positive reactions across all rounds; presenting the aggregated positive-reaction candidates to the user in a side-by-side comparison view; receiving a final selection from the user; and storing the selected name as the session's chosen result associated with the user's session chain.

Claim 15. A computer-implemented system comprising: a taste state data structure per session chain containing inferred preference dimensions each associated with a confidence level, an uncertainty flag, a strength-of-evidence counter, and positive and negative signal inventories; an evidence extractor that, upon

receiving a user reaction to a generated name candidate, extracts phonetic, stylistic, and territorial signals from the name and updates the taste state by adjusting confidence levels, signal inventories, and contradiction records associated with the matched preference dimensions; wherein strong positive reactions increase dimension confidence more than mild positive reactions, and strong negative reactions decrease dimension confidence more than mild negative reactions.

Claim 16. The system of claim 15, further comprising an exploration planner that reads the taste state before each generation round and produces an exploration strategy specifying: a set of target preference dimensions selected based on uncertainty level and unexplored territory priority; an exploration ratio governing the fraction of generated name candidates to be produced under exploratory directives versus exploitative directives aligned with high-confidence preferences; and probe directives specifying phonetic, stylistic, or territorial characteristics for exploratory candidates; wherein the exploration strategy is provided to the brief constructor and quality adapter to influence candidate generation and scoring.

Claim 17. A computer-implemented method comprising: maintaining a taste state for a naming session chain, the taste state comprising confidence levels, uncertainty flags, contradictory evidence records, and an unexplored preference territory map; before each generation round, invoking an exploration planner to read the taste state and produce an exploration strategy; generating name candidates under the exploration strategy; presenting candidates to a user and capturing reactions; invoking an evidence extractor to update the taste state based on the reactions; and repeating the exploration planning, generation, presentation, reaction capture, and taste state update cycle for each subsequent round in the session chain; wherein the taste state evolves across rounds to converge on a model of the user's naming preferences.

ABSTRACT

A computer-implemented name generation platform elicits user preferences through a guided sequential intake process that presents one question at a time, collects responses into a direction profile, presents the direction for review and editing, and normalizes confirmed preferences into a versioned canonical naming intent object. The canonical intent drives a naming brief supplied to a multi-provider AI generation router that dispatches requests to multiple language model providers, scores candidates using vertical-specific quality adapters with weighted dimension scoring, and selects highest-quality results. User reactions to presented name candidates are analyzed to construct a taste profile encoding liked and disliked phonetic patterns, stylistic territories, and preference scores. The taste profile augments subsequent generation rounds in a structured multi-round refinement journey implemented through a session chain architecture in which each round creates a new session record linked to the prior round, enabling replay, taste reconstruction, round completion enforcement, and exclusion of previously presented names. A `priority_weights` field in the canonical naming intent propagates user-specified priorities into both quality adapter scoring and AI prompt construction. A versioned intake schema migration system stores field definitions, allowed values, deprecated aliases, and migration rules, and translates stored intake data to the current schema version without re-collecting user input. The platform operates across multiple distinct naming domains (baby names, pet names, business names, and others) through a configuration-driven vertical registration system, applying identical platform behavior with domain-specific intake questions, AI prompt context, quality scoring criteria,

and visual presentation. In an advanced embodiment, the system maintains a persistent Taste State containing inferred preference dimensions with confidence levels, uncertainty flags, and contradictory evidence records, and employs an Exploration Planner to actively direct generation rounds toward resolving uncertainty and probing unexplored preference territories, creating a self-improving preference model that converges on the user's naming taste over successive rounds.

INVENTOR DECLARATION

(To be completed at time of filing)

I hereby declare that:

- I am the original inventor of the subject matter claimed in the above provisional patent application.
- I have reviewed and understand the contents of this application.
- The duty to disclose information material to patentability exists under 37 CFR 1.56.

Inventor Name: _____

Signature: _____

Date: _____

Address: _____

FILING INSTRUCTIONS

See the separate step-by-step filing guide below.